

Pipeline Dataverse - Archivematica

Manuale d'uso — Pipeline Dataverse -> Archivematica

Questo manuale documenta l'uso di due script Python che automatizzano il trasferimento di dataset da **Dataverse UNIMI** (dataverse.unimi.it) ad **Archivematica** per la conservazione a lungo termine.

Script	Funzione
<code>scarica_dataverse.py</code>	Scarica dataset e metadati da Dataverse e li organizza in pacchetti compatibili con Archivematica
<code>archivematica_ingest.py</code>	Trasferisce ed esegue l'ingest dei pacchetti in Archivematica, tenendo traccia di cio' che e' gia' stato processato

Indice

1. [Panoramica del flusso](#)
2. [Prerequisiti](#)
3. [Configurazione — file .env](#)
4. [Script 1 — `scarica\_dataverse.py`](#)
5. [Script 2 — `archivematica\_ingest.py`](#)
6. [Struttura del pacchetto generato](#)
7. [File di stato e ripresa dopo interruzioni](#)
8. [Risoluzione dei problemi comuni](#)
9. [Esempio di esecuzione completa](#)

1. Panoramica del flusso

```
dois.txt                               scarica_dataverse.py           Transfer
Source
(lista DOI)  ----->  - scarica file             -----> Location
                  - scarica metadati
(Archivematica)
                  - genera metadata.csv             |
                  - genera dcat.json (opt.)          v
                  - scarica schema.json (opt.)
archivematica_ingest.py
processingMCP.xml                      - copia
transfer                               - avvia
transfer                               - auto-approva
ingest                                 - attende
UUID                                   - salva AIP
```

Il flusso tipico e':

1. Si prepara `dois.txt` con l'elenco dei DOI da archiviare.
 2. Si esegue `scarica_dataverse.py`, che scarica tutte le versioni di ciascun dataset direttamente nella **Transfer Source Location** di Archivematica.
 3. Si esegue `archivematica_ingest.py`, che avvia il transfer e l'ingest per ogni pacchetto non ancora processato.
 4. Lo stato di ogni pacchetto viene salvato in `stato_archivematica.json`.
-

2. Prerequisiti

Software

```
pip install requests --break-system-packages
```

Accesso a Dataverse

- Una **API key Dataverse** (necessaria solo per dataset ad accesso ristretto; per dataset pubblici può essere lasciata vuota).

Accesso ad Archivematica

- **Dashboard API key** dell'utente Archivematica (es. zfed)
- **Storage Service API key** dell'utente Storage Service (es. archivematica)
- **UUID della Transfer Source Location**
- Accesso in lettura/scrittura alla Transfer Source Location sul filesystem

Permessi filesystem

```
sudo usermod -aG archivematica <utente>  
# Richiede logout/login per avere effetto
```

Note su SSL

Se Archivematica usa un certificato self-signed, impostare `SSL_VERIFY=false` nel `.env` o usare `--no-verify-ssl` da riga di comando.

URL tipici: - Dashboard: `https://<host>` (porta 443) - Storage Service: `https://<host>:8000`

3. Configurazione — file `.env`

```
cp .env.template .env  
nano .env
```

Contenuto del `.env`

```
# Dataverse  
DATAVERSE_API_KEY=<chiave_dataverse>  
EXPORT_DCAT=false          # true per generare dcat.json e  
schema.json  
  
# Archivematica Dashboard  
AM_URL=https://192.168.139.39  
AM_USER=zfed  
AM_API_KEY=<chiave_dashboard>  
  
# Archivematica Storage Service  
SS_URL=https://192.168.139.39:8000
```

```

SS_USER=archivematica
SS_API_KEY=<chiave_storage_service>

# Transfer
AM_TRANSFER_SOURCE_UUID=<uuid>
AM_TRANSFER_TYPE=standard
AM_PROCESSING_CONFIG=dataverse_001
AM_AUTO_APPROVE=true

# SSL
SSL_VERIFY=false

```

Recuperare i valori mancanti

```

# UUID Transfer Source
python3 archivematica_ingest.py --no-verify-ssl --list-sources

# Processing configuration disponibili
python3 archivematica_ingest.py --no-verify-ssl --list-processing-
configs

```

Errori comuni nel .env

Errore	Sintomo	Correzione
SS_URL=https://...	MissingSchema	SS_URL=https://...
SS_URL=http://...	302 Found	SS_URL=https://...
Chiavi API vuote	401 Unauthorized	Compilare le chiavi API
UUID errato	404 Not Found	Usare --list-sources
Variabili esportate in sessione precedente	Valori vecchi	Nuova sessione shell

Priorita' delle variabili

1. Argomento CLI (es. --apikey)
2. Variabile d'ambiente di sistema (export)
3. File .env
4. Default nel codice

Sicurezza

```
echo ".env" >> .gitignore
```

4. Script 1 — scarica_dataverse.py

Cosa fa

Per ogni DOI elencato in dois.txt:

1. Recupera **tutte le versioni** pubblicate del dataset
2. Per ciascuna versione:
 - scarica tutti i file del dataset
 - salva i metadati Dataverse in dataverse.json
 - genera metadata.csv Dublin Core per quella versione
 - genera dcat.json (DCAT-AP JSON-LD) se EXPORT_DCAT=true
 - scarica schema.json (Schema.org JSON-LD) se EXPORT_DCAT=true
3. Genera un metadata.csv complessivo nella radice del pacchetto per Archivematica (METS dmdSec)

Esportazione DCAT-AP e Schema.org

Quando EXPORT_DCAT=true (o --dcat da CLI), per ogni versione vengono generati due file aggiuntivi nella cartella metadata/ della versione:

dcat.json — DCAT-AP 2.x JSON-LD generato **localmente** dai metadati già scaricati, senza chiamate HTTP aggiuntive. Include:

Campo DCAT-AP	Sorgente Dataverse
dct:title	title
dct:creator	author (nome + affiliazione)
dcat:contactPoint	datasetContact (vCard)
dct:description	dsDescription
dcat:theme	subject
dcat:keyword	keyword
dct:issued	productionDate / distributionDate
dct:publisher	publisher
dct:license	license / termsOfUse
dcat:distribution	file del dataset (formato, dimensione, checksum MD5)
owl:versionInfo	numero versione (es. 1.0)

schema.json — Schema.org JSON-LD scaricato direttamente dall'API Dataverse tramite l'exporter schema.org.

Opzioni da riga di comando

Opzione	Default	Descrizione
--dois <file>	dois.txt	File con l'elenco dei DOI
--output <cartella>	DATASET TRASFERITI	Cartella di destinazione
--apikey <chiave>	dal .env	API key Dataverse
--no-verify-ssl	false	Disabilita verifica SSL
--dcat	false	Abilita esportazione DCAT-AP e Schema.org

Esempi

```
# Standard
python3 scarica_dataverse.py

# Con DCAT-AP e Schema.org
python3 scarica_dataverse.py --dcat

# Specificando l'output
python3 scarica_dataverse.py \
  --output /mnt/e/DROPBOX/Dropbox/_ARKIVE/TRANSFER_SOURCE \
  --dcat
```

Output a schermo

```
=====
Dataverse UNIMI -- Download automatico (tutte le versioni)
Sorgente DOI : dois.txt (3 DOI)
Destinazione : /mnt/.../TRANSFER_SOURCE
API key      : *****5678
SSL verify   : False
Export DCAT  : True
=====
```

```
[1/3] DOI: doi:10.13130/RD_UNIMI/KS2PUX
--> Recupero lista versioni...
--> 1 versione/i trovata/e: v1.0
-- Versione v1.0 [RELEASED]
[metadata] dataverse.json salvato
[metadata] dcat.json generato
[metadata] schema.json salvato
[data] 4 file trovati, inizio download...
Scarico: works.tab ... OK (25 KB)
[metadata] metadata.csv (versione) generato (1 righe)
[metadata] metadata.csv generato (1 righe, 1 versione/i)
[validazione] metadata.csv OK
OK Stato: ok | Versioni: 1 | File OK: 4 | Errori: 0
```

Comportamento idempotente

- File già scaricati: **saltati**
- dataverse.json, dcat.json, schema.json, metadata.csv di versione: generati solo se mancanti
- metadata.csv complessivo: **sempre rigenerato**

5. Script 2 — archivematica_ingest.py

Cosa fa

Per ogni pacchetto DOI nella Transfer Source Location non ancora completato:

1. Copia processingMCP.xml nella radice del pacchetto
2. Avvia il **Transfer** con nome AIP-<YYYYMMDD>-<DOI>
3. Approva automaticamente il transfer tramite API (se AM_AUTO_APPROVE=true)
4. Attende il completamento del Transfer
5. Attende il completamento dell'**Ingest** (SIP -> AIP)
6. Salva l'UUID dell'AIP nel file di stato

Come funziona l'auto-approve

Lo script interroga /api/transfer/unapproved/ per trovare i transfer in attesa, identifica quello corrispondente tramite il nome, e lo approva via /api/transfer/approve/. Elimina la necessità di cliccare manualmente "Approve transfer" nel Dashboard.

Opzioni da riga di comando

Sorgente e stato

Opzione	Default	Descrizione
--source <cartella>	da Transfer Source	Directory con i pacchetti
--state-file <file>	stato_archivematica.json	File di tracciamento

Dashboard

Opzione	Default	Descrizione
--am-url <url>	dal .env	URL Dashboard
--am-user <utente>	dal .env	Utente Dashboard
--am-apikey <chiave>	dal .env	API key Dashboard

Storage Service

Opzione	Default	Descrizione
--ss-url <url>	dal .env	URL Storage Service
--ss-user <utente>	dal .env	Utente Storage Service
--ss-apikey <chiave>	dal .env	API key Storage Service

Transfer

Opzione	Default	Descrizione
--transfer-source <UUID>	dal .env	UUID Transfer Source Location
--transfer-type <tipo>	standard	Tipo di transfer
--processing-config <nome>	dataverse_001	Processing configuration
--auto-approve	dal .env	Approva automaticamente

Utility

Opzione	Descrizione
--list-sources	Elenca Transfer Source Locations ed esce
--list-processing-configs	Elenca processing configuration ed esce
--no-verify-ssl	Disabilita verifica SSL
--poll-interval <sec>	Intervallo polling (default: 15s)
--dry-run	Mostra cosa farebbe senza eseguire

Esempi

```
# Standard (tutto da .env)
python3 archivematica_ingest.py

# Lista Transfer Source
python3 archivematica_ingest.py --no-verify-ssl --list-sources

# Anteprima
python3 archivematica_ingest.py --dry-run
```

6. Struttura del pacchetto generato

```
TRANSFER_SOURCE/
doi_10.13130_RD_UNIMI_KS2PUX/
objects/
v1.0/
objects/                <- file del dataset
works.tab
README.txt
metadata/               <- metadati di questa versione
dataverse.json          <- metadati Dataverse
(preservation)
metadata.csv            <- Dublin Core di questa
versione
dcat.json               <- DCAT-AP JSON-LD (solo se
EXPORT_DCAT=true)
schema.json             <- Schema.org JSON-LD (solo se
EXPORT_DCAT=true)
v2.0/
```

```

objects/
  works_v2.tab
metadata/
  dataverse.json
  metadata.csv
  dcat.json          <- (solo se EXPORT_DCAT=true)
  schema.json        <- (solo se EXPORT_DCAT=true)
metadata/
  metadata.csv       <- Dublin Core TUTTE le versioni
-> METS dmdSec
  processingMCP.xml  <- copiato da
archivematica_ingest.py

```

Formato metadata.csv (radice)

filename	dc.title	dc.creator	dc.date	
objects/v1.0/objects/works.tab	Titolo	Autore	2026-03-04	c
objects/v2.0/objects/works_v2.tab	Titolo	Autore	2026-03-04	c

7. File di stato e ripresa dopo interruzioni

stato_archivematica.json

```

{
  "doi_10.13130_RD_UNIMI_KS2PUX": {
    "transfer_name": "AIP-20260619-doi_10.13130_RD_UNIMI_KS2PUX",
    "transfer_uuid": "bde477ff-...",
    "sip_uuid": "a1b2c3d4-...",
    "aip_uuid": "e5f6a7b8-...",
    "status": "ingested",
    "completed_at": "2026-06-19T10:42:13+00:00",
    "error": null
  }
}

```

Valori di status

Valore	Significato
in_progress	Avviato ma non completato
transferred	Transfer ok, ingest non completato
ingested	Completato - saltato nelle esecuzioni successive
failed	Errore - ritentato da zero al prossimo avvio

Reset manuale di un pacchetto

```

python3 -c "
import json
s = json.load(open('stato_archivematica.json'))
s.pop('doi_10.13130_RD_UNIMI_KS2PUX', None)
json.dump(s, open('stato_archivematica.json', 'w'), indent=2)
"

```

Reset completo

```

echo '{}' > stato_archivematica.json

```

8. Risoluzione dei problemi comuni

MissingSchema / URL non valido

MissingSchema: Invalid URL 'https://...'

Aggiungere i due punti: https:// non https//.

302 Found sulla porta 8000

```
SS_URL=https://192.168.139.39:8000
```

401 Unauthorized

Verificare AM_API_KEY e SS_API_KEY nel .env.

404 Not Found sulla Transfer Source UUID

```
python3 archivematica_ingest.py --no-verify-ssl --list-sources
```

SSL Certificate Verify Failed

```
SSL_VERIFY=false    # nel .env  
# oppure  
python3 archivematica_ingest.py --no-verify-ssl
```

Permission Denied su processingMCPConfigs

```
sudo usermod -aG archivematica <utente>
```

Transfer REJECTED al riavvio

Lo script rileva automaticamente status failed e riparte da zero.

400 “Unable to determine the status of the unit”

Errore transitorio: lo script ritenta automaticamente 2 volte (20s di attesa).

Auto-approve non funziona

1. Verificare AM_AUTO_APPROVE=true nel .env
2. Verificare che il transfer sia in USER_INPUT nel Dashboard
3. Aumentare APPROVE_TRANSFER_WAIT nel codice (default: 10s)

Elasticsearch non raggiungibile

```
sudo systemctl start elasticsearch  
curl -s http://localhost:9200
```

9. Esempio di esecuzione completa

Configurazione iniziale (una tantum)

```
git clone https://github.com/zfed/Arkive.git  
cd Arkive/tools/dataverse-archivematica  
  
pip install requests --break-system-packages  
  
cp .env.template .env
```

```
nano .env

# Recupera UUID Transfer Source
python3 archivematica_ingest.py --no-verify-ssl --list-sources
# Aggiorna AM_TRANSFER_SOURCE_UUID nel .env

# Verifica processing configuration
python3 archivematica_ingest.py --no-verify-ssl --list-processing-
configs
```

Esecuzione standard

```
# 1. Lista DOI da archiviare
cat > dois.txt << 'EOF'
doi:10.13130/RD_UNIMI/KS2PUX
doi:10.13130/RD_UNIMI/NFURUT
EOF

# 2. Scarica i dataset (con metadati DCAT-AP opzionali)
python3 scarica_dataverse.py \
  --output /mnt/e/DROPBOX/Dropbox/_ARKIVE/TRANSFER_SOURCE \
  --dcat

# 3. Ingesta in Archivematica (completamente automatico)
python3 archivematica_ingest.py

# 4. Verifica stato
cat stato_archivematica.json | python3 -m json.tool
```

Per le esecuzioni successive ripetere i passi 2 e 3: gli script elaborano solo cio' che e' nuovo o non ancora completato.